3.7 ppt

3.7 项目演进路线 阶段三：Codebase Agent v3

开场：从学会机制，到交付项目

- 项目项目二：让模型提出的工具调用受控执行。
- 项目阶段三：让 Agent 连续思考、执行、观察、纠错和结束。
- 项目目标：组合 Loop、Planning、State、Checkpoint、Sandbox、Harness 与审批流程。
- 两类任务：

  任务一：代码理解

  > *“帮我找出该项目的登录逻辑，并生成 Markdown 说明文档。”*

  Agent 能够：

  - 搜索认证相关代码
  - 阅读路由、中间件、Service、数据库逻辑
  - 梳理调用链
  - 输出模块说明
  - 给出相关文件路径

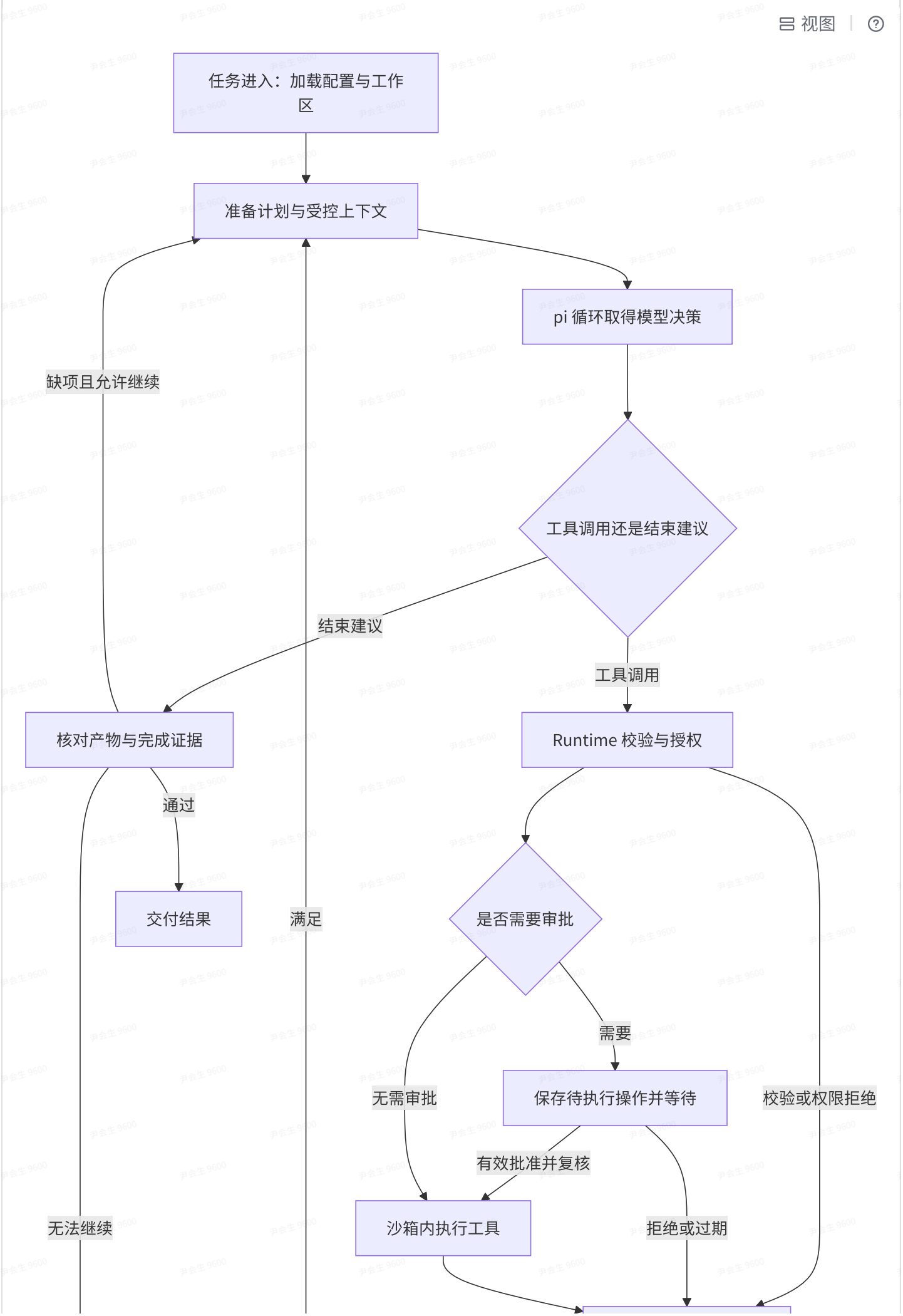
  任务二：测试生成与修复

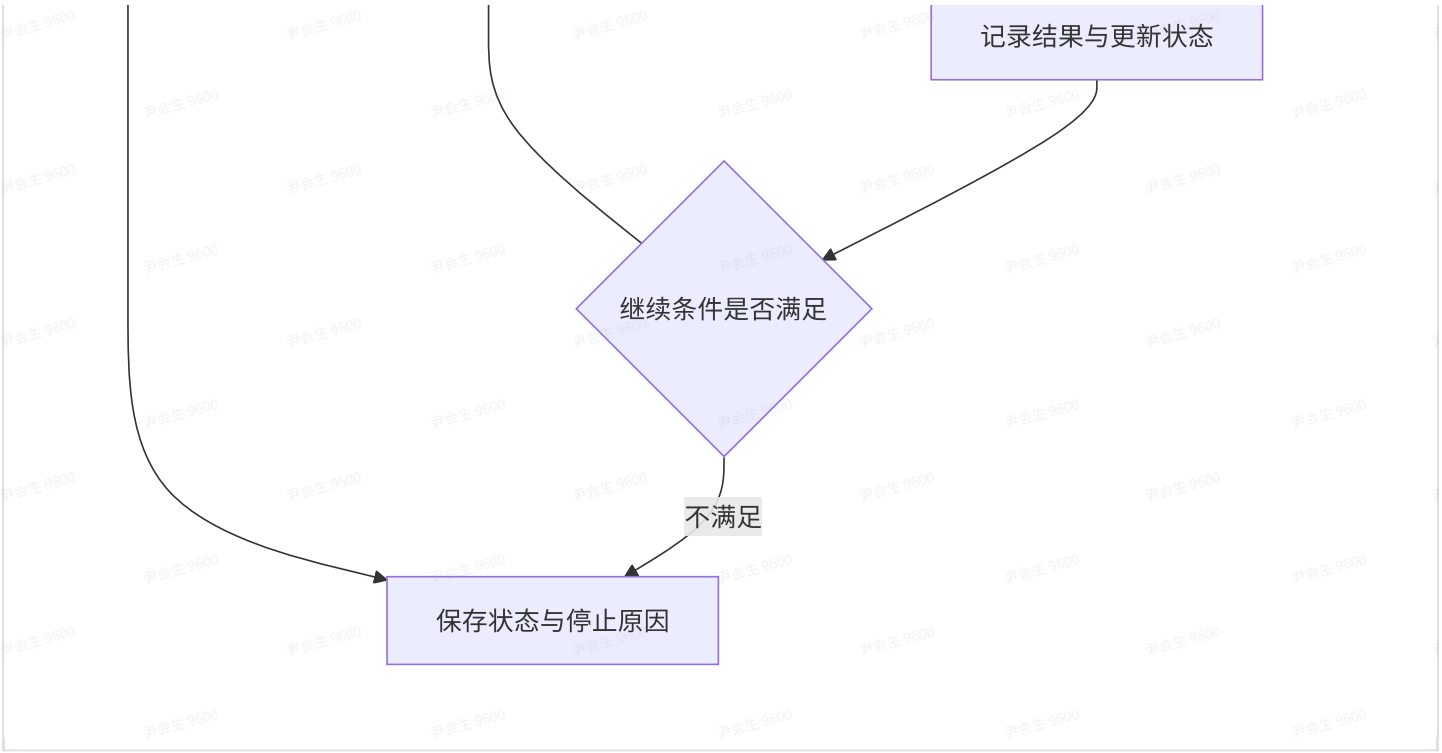
  > *“给指定函数补充测试，并根据测试失败日志进行修复。”*

  Agent 能够：

  - 定位函数
  - 分析已有测试、编写测试、执行测试
  - 读取失败日志
  - 修正测试或代码
  - 输出变更说明

- 核心要求：可运行、可验证、可恢复、可继续演进。


视图
任务进入：加载配置与工作区
准备计划与受控上下文
pi 循环取得模型决策
工具调用还是结束建议
结束建议
工具调用
缺项且允许继续
核对产物与完成证据
Runtime 校验与授权
通过
交付结果
满足
是否需要审批
需要
无需审批
保存待执行操作并等待
校验或权限拒绝
有效批准并复核
沙箱内执行工具
拒绝或过期
无法继续



一、项目目标盘点：我们已经有什么，还缺什么？

1.1 把第三章目标拆成可验收能力

能力	可验收表现
任务理解与规划	能把目标拆成有顺序、有完成条件的步骤
仓库探索	能列出文件、搜索代码、读取相关文件，并给出路径依据
文件修改	能按任务创建或修改文件，并通过差异确认改动范围
测试与纠错	能运行测试、读取失败结果，并据此决定下一步
Loop 与停止	能把工具结果带回下一轮；达到完成条件或预算时停止
人工确认	高风险操作前能暂停，让人查看操作内容后再继续
安全执行	文件访问和命令执行受工作区边界约束
状态保存与恢复	中断后能找回任务进度，并从可确认的位置继续
执行编排	能明确步骤的先后、分支、循环和责任组件
过程追踪	能查看工具调用、执行结果、任务状态和最终改动

- 验收重点：系统实际做了什么。
- 完成依据：产物、测试、差异与执行记录。

1.2 标出已实现、部分实现和待补齐

项目能力	前面章节	目前基础	本项目状态与处理方式
连续思考、执行、观察与纠错	3.1 Agent Loop	已有 Loop 的执行模型和停止控制思路	**部分实现**：需要接入代码仓库工具，并用两类任务验证闭环
任务拆解与步骤规划	3.2 Planning	已有计划结构、依赖和偏差处理方法	**部分实现**：需要把计划用于代码理解和测试修复流程
任务状态与 Checkpoint	3.3 State Machine 与 Checkpoint	已有状态设计、存档和恢复思路	**部分实现**：需要确认状态中保存了代码任务恢复所需的信息
文件与命令执行边界	3.4 Sandbox	已有沙箱职责和边界设计	**部分实现**：需要核对项目工具是否都经过边界检查
执行流程与人工协作	3.5 Harness	已有任务装配、执行、审查和异常处理方法	**部分实现**：需要把它们连接到本项目的完整任务路径
节点、分支、循环与恢复	3.6 LangGraph	已有图式编排方法及其适用场景	**可复用的方法**：是否由 LangGraph 承担编排，要结合本项目结构判断
仓库工具	第三章项目目标	已明确需要十件工具	**需要补齐或接入**：文件列表、代码搜索、文件读写、补丁、测试、命令和 Git 状态等
两类典型任务	第三章项目目标	已明确代码理解、测试与修复的业务目标	**需要组合验收**：用真实样本仓库验证从输入到产出的完整过程

- 学过机制 ≠ 已接入项目。
- 盘点以当前工程的可运行能力为准。

1.3 阶段二的 Runtime 能力如何被复用

阶段二 Runtime 能力	在 Codebase Agent 中的对应方式
ToolRegistry 与工具定义	注册十件仓库工具，并描述工具用途和输入参数
JSON Schema / Pydantic 校验	检查工具参数格式，避免缺字段或类型错误进入执行函数
权限与风险分级	区分读取、修改文件、运行命令等不同风险的操作
人工确认	对需要审批的操作暂停执行，并提供可检查的信息
超时、重试与降级	为工具执行设置边界；根据错误类型决定是否重试或交由人工处理
审计与 Trace	记录谁发起了什么工具调用、参数是什么、结果如何
MCP 接入经验	复用统一接入和工具描述的设计思路；本节的重点则是本地仓库工具

- 复用工具治理原则与执行路径。
- 根据仓库任务补齐工具、策略与验收条件。

1.4 本节的交付范围与验收标准

- 十件仓库工具全部可用。
- 两类任务完成端到端验收。
- 参数经过校验，操作受执行边界约束。
- 测试失败后依据真实日志调整行动。
- 审批前暂停，批准后执行对应操作。
- 恢复时核对现场，避免盲目重放写操作。
- Trace、状态与 Git 差异支持复核。
- 失败原因清楚、可定位、可恢复。

二、先跑通基线：Agent 能不能完成两类仓库任务？

- 先看结果，再组装能力。
- 观察：任务入口、工具推进、完成证据。
- 基线：`project3/v2`。
- TypeScript：Agent 与工具运行时。
- Python：LangGraph 审批服务。

2.1 基线项目结构与运行方式

路径	职责	代码作用
v2/src/cli.ts	命令行入口	提交任务、处理审批、查询审计
v2/src/pi/provider.ts	模型接入	本次任务是否连接了真实模型
v2/config/platform.yaml	平台配置	模型、预算、沙箱等公共约束
v2/config/tasks.yaml	任务配置	任务差异、审批方式与审批人角色
v2/hitl/service.py	LangGraph 审批服务	等待审批、接收决定并推动后续执行
samples/1_login_demo	代码理解样本	登录逻辑及生成的说明文档
samples/2_bug_fix/demo-app	测试修复样本	失败测试、源码改动与复测结果
v2/data/platform.db	本地运行账本	审批记录、审计与 Trace

- Agent 工程组织执行，样本仓库提供操作对象。
- `--dir` 指定工作仓库。

```
cd project3/v2
```

- 前提：依赖已安装，`DEEPSEEK_API_KEY` 已配置。
- 启动时检查决策器标识。
- 离线脚本验证工程机制；真实模型验证任务能力。
- `trace_id`：标识一次运行。
- 审批编号：标识一次待批准操作。

2.2 任务一：理解登录逻辑并生成说明

代码块

```
node --import tsx src/cli.ts agent \
  "帮我找出该项目的登录逻辑，并生成 Markdown 说明文档。" \
  --dir ../samples/1_login_demo
```

- 目录探索 → 搜索认证代码 → 阅读入口 → 追踪调用关系 → 生成说明。
- 先取得代码依据，再形成结论。
- 样本不存在的模块，不补写、不编造。
- 基线写入文档时返回 `pending_approval_id`。

代码块

```
node --import tsx src/cli.ts approvals decide <pending_approval_id> \
  --decision approve \
  --actor trainer-1 \
  --roles maintainer \
  --task understand
```

代码块

```
node --import tsx src/cli.ts approvals execute <pending_approval_id> \
  --task understand
```

- `decide`：记录授权。
- `execute`：执行获批操作。
- 批准成功后，仍需检查文件是否生成。

代码块

```
cat ../samples/1_login_demo/artifacts/login-flow.md
```

代码块

```
node --import tsx src/cli.ts audit --trace-id <trace_id>
```

检查项	要确认什么
登录入口	能否指出真实的入口文件与处理位置
调用关系	说明中的调用顺序是否有代码依据
路径引用	引用的文件是否存在，是否与所述逻辑对应
文档落盘	Markdown 文件是否实际生成
修改范围	除预期文档外，是否出现无关改动

2.3 任务二：补测试、读失败日志并修复

- 基线预置失败测试，验证读取失败、修改源码与复测。
- 已有测试禁止修改；新增测试限定 `tests/generated/`。
- 样本缺陷：会话到期判断只比较日期，未完整比较时间。

代码块

```
npm --prefix ../samples/2_bug_fix/demo-app test --silent
```

代码块

```
ℹ tests 7
ℹ pass 4
ℹ fail 3
```

先启动审批服务

代码块

```
HITL_API_KEY=demo-key HITL_DB=/tmp/hitl-demo.db \
  ../.venv/bin/python -u hitl/service.py
```

代码块

```
curl -s \
  -H "Authorization: Bearer demo-key" \
  http://127.0.0.1:8383/health
```

- `demo-key` 为本地演示值，两端保持一致。

再提交修复任务

代码块

```
HITL_API_KEY=demo-key node --import tsx src/cli.ts agent \
  "读取登录会话模块的失败测试与日志，修复源码并复测，不修改现有测试。" \
  --task fix_tests \
  --dir ../samples/2_bug_fix/demo-app
```

- 测试 → 失败分析 → 提出修改 → 审批 → 执行 → 复测 → 汇总。
- `run_test` 执行仓库脚本，按任务策略审批。

代码块

```
HITL_API_KEY=demo-key node --import tsx src/cli.ts \
  approvals tickets --task fix_tests
```

代码块

```
HITL_API_KEY=demo-key node --import tsx src/cli.ts \
  approvals decide <ticketId> \
  --decision approve \
  --actor lead-1 \
  --roles tech_lead \
  --task fix_tests
```

- HTTP 审批路径：任务等待批准，获批后继续。
- 每张新审批单对应一次新操作。

最后检查结果与改动

代码块

```
npm --prefix ../samples/2_bug_fix/demo-app test --silent
```

代码块

```
git -C ../samples/2_bug_fix/demo-app diff
```

代码块

```
git -C ../samples/2_bug_fix/demo-app status --short
```

- 样本参考结果：七条测试通过，日期比较改为时间戳比较。
- 核对失败原因、源码差异、测试完整性与变更说明。
- 修复闭环通过，不代表新增测试能力已通过。
- 新用例必须证明能暴露目标缺陷。

2.4 观察基线：哪些能力已经就绪，哪些还不够？

观察点	当前基线表现	后续要补齐或验证什么
任务配置	已有任务配置与审批配置	查清计划、路径、命令是否仍有任务专用逻辑散落在代码中
工具参数契约	已有阶段二 Runtime 基础	核对十件仓库工具是否统一经过校验，失败信息是否进入下一轮
测试生成	当前样本预置测试，并禁止改动测试目录	增加受控的新测试写入范围，证明新测试能发现目标缺陷
上下文管理	搜索、源码和测试日志都会进入任务过程	检查输出过长时如何截断、保留证据并控制模型输入
人工确认	本地审批和 HTTP 审批的继续方式不同	明确批准后由谁执行、由谁继续任务，避免漏执行或重复执行
完成判定	模型可能先于实际执行宣布完成	以产物、测试、差异和审计记录核对完成状态
批量验收	当前 README 按两个样本分别运行	汇总两类任务的结果、停止原因、审批与产物，形成可复核报告

- 新增测试受控写入。
- 禁止删除断言、跳过用例或降低已有标准。

三、未来接入新任务，改动落在哪里？

变化类型	改动位置
已有工具能完成的任务	任务配置 config/tasks.yaml
需要新工具或新验证方式的任务	扩展代码

- 配置描述目标、交付物、权限、路径、预算与审批。
- 模型根据观察决定具体文件与下一步动作。
- 检查 Markdown TODO：复用工具，通过配置接入。
- 检查网页链接：需要网络工具，扩展代码。

```
代码块
1  if (taskType === "docs_audit") {
2    // 任务专用逻辑
3  }
```

- 任务专用分支散落在 Loop、工具和审批模块时，检查职责边界。
- 任务差异进入配置，新能力扩展代码。
- 三条公共边界：参数契约、上下文预算、人工确认。

四、让工具调用有契约

- 模型展示与执行校验使用同一份工具定义。
- 参数不合法，不进入执行。
- 错误准确回写，支持下一轮修正。

4.1 十件仓库工具各自的输入与输出

工具	输入应说明什么	输出应包含什么
list_files	相对目录、匹配条件、数量上限	文件路径列表，是否截断
search_code	搜索内容、搜索范围、结果上限	命中文件、行号、片段，是否截断
read_file	文件路径、可选的起始行与读取范围	文件内容、实际行号范围，是否截断
read_directory	目录路径	当前目录的直接子项及类型
write_file	文件路径、写入内容、覆盖规则	写入路径、创建或覆盖结果
apply_patch	目标文件、补丁内容	应用结果、变更摘要；失败时说明匹配问题
run_test	允许的测试目标或配置标识	退出码、测试摘要、输出日志，是否超时
run_command	命令及参数、工作目录、超时	退出码、标准输出、标准错误，是否超时
git_diff	可选的路径范围	实际差异、涉及的文件，是否截断
git_status	可选的路径范围	已修改、未跟踪等文件状态

- 实现位置：`project3/v2/src/tools/`。

- `read_directory`：查看一层目录；`list_files`：按范围查找文件。
- `run_test`：解析允许的测试配置；`run_command`：单独检查命令范围。
- 搜索无匹配与搜索失败分别表达。
- 工具执行成功与任务目标达成分别表达。
- 输出同时服务模型决策与程序记账。

4.2 用 JSON Schema 校验工具参数

代码块

```
{
  "type": "object",
  "properties": {
    "path": {
      "type": "string",
      "minLength": 1,
      "description": "相对于当前工作仓库的文件路径"
    },
    "start_line": {
      "type": "integer",
      "minimum": 1
    },
    "max_lines": {
      "type": "integer",
      "minimum": 1,
      "maximum": 500
    }
  },
  "required": ["path"],
  "additionalProperties": false
}
```

- 检查必填字段、类型、数值范围与未知字段。
- Schema 校验通过后，继续检查权限与执行边界。

检查层	回答的问题
参数校验	字段、类型、范围是否符合工具契约
权限与审批	当前任务和用户是否允许执行这次操作
Sandbox	实际文件访问与进程执行是否受到边界约束
工具执行	文件是否存在、补丁是否匹配、命令能否完成

- 模型发现与 Runtime 校验绑定同一版本。
- 参数错误显式返回，避免静默类型转换。

4.3 校验失败后如何反馈、修正和重试

代码块

```
{
  "path": 42
}
```

代码块

```
{
  "ok": false,
  "error": {
    "code": "INVALID_ARGUMENT",
    "message": "path 必须是字符串，当前收到整数",
    "details": {
      "field": "path",
      "expected": "string",
      "received": "integer"
    }
  },
  "execution": {
    "started": false
  }
}
```

- `started: false`：确认执行尚未发生。
- 保留 `toolCallId` / `tool_call_id`，关联原调用。
- 模型修正后提交新调用，重新经过完整检查。

代码块

```
{
  "path": "src/routes/login.ts",
  "start_line": 1,
  "max_lines": 120
}
```

情况	应怎样处理
参数不合法，执行尚未开始	把错误交给模型，让它修正参数后重新提交
参数合法，但执行遇到临时故障	根据第二章的重试策略，判断是否允许有限重试
写操作发出后结果不明	先检查执行事实，确认是否已经产生修改

- 参数错误需要修正，原样重试无效。
- 写入内容变化，重新检查审批。
- 重复错误受循环预算与护栏约束。

4.4 验收：错误参数不能进入工具执行

验收场景	预期结果	必须检查的执行事实
read_file 缺少 path	返回参数错误，指出缺失字段	handler 调用次数为 0
read_file 的 path 为数字	返回类型错误	没有发生文件读取
write_file 缺少 content	返回参数错误	文件未创建，已有内容未改变
参数包含未定义字段	按严格契约拒绝	handler 未执行
模型修正参数后重新提交	新调用通过校验并执行	成功结果关联到新的调用编号
连续提交相同错误参数	达到既有护栏条件后停止	没有无限请求或工具执行

代码块

```
{
  "path": "../../outside.txt"
}
```

- 类型正确，仍应被工作区边界拒绝。
- 验证错误来自正确检查层，未读取外部内容。
- 修正流程：错误回写 → 编号关联 → 完整复检 → 合法执行一次。
- 工程师复核：定义来源、校验位置、错误回写、副作用阻止。

五、控制长任务的上下文与预算

- 完整执行记录用于追踪。
- 每轮模型输入经过选择与预算检查。

- 两项机制：截断单次观察，压缩较早步骤。

5.1 为什么代码任务更容易撑满上下文

执行步骤	新增内容	后续真正需要的信息
搜索登录逻辑	多处命中及代码片段	相关文件和入口位置
阅读实现	整个文件的源码	与缺陷有关的函数和依赖
第一次测试	成功日志、失败堆栈、统计	失败用例、错误位置、预期与实际值
应用补丁	修改内容与执行结果	改了哪里、是否应用成功
第二次测试	又一份完整日志	当前是否通过、还有什么失败

- 工具输出远大于用户输入，历史持续增长。
- 单轮预算：系统提示词、工具定义、任务与历史，并预留输出空间。
- 累计预算：Token、费用、工具调用次数与运行时间。
- 压缩降低后续输入，既有消耗仍计入账本。

5.2 截断单次工具观察

工具结果	优先保留	内容过长时怎样处理
文件列表	路径、数量、是否还有结果	限制条数，提示缩小目录或继续查询
搜索结果	命中文件、行号、相关片段	限制命中数，明确结果不完整
文件内容	请求范围内的源码与行号	分段读取，返回本次实际范围
测试日志	退出码、失败用例、关键堆栈、统计	完整日志落盘，提供可继续读取的引用
Git 差异	文件列表、变更统计、相关补丁	按文件读取，提示还有未展示的差异

- 完整结果保存为受控产物。
- 模型观察保留关键状态、截断标记与补查入口。
- 测试日志优先提取失败信息；提取失败时保留有限首尾片段。

代码块

```
{
  "exit_code": 1,
  "timed_out": false,
  "summary": "7 条测试，4 条通过，3 条失败",
  "failures": [
    {
      "test": "会话到期后应失效",
      "location": "tests/session-policy.test.ts:42",
```

```
      "message": "预期 false，实际 true"
    }
  ],
  "truncated": true,
  "output_ref": "artifacts/runs/run-001/test-01.log"
}
```

- 示例结构，非真实运行记录。
- `output_ref` 必须能通过受控读取路径访问。

5.3 压缩历史步骤，保留关键证据

内容	处理方式
用户目标、禁止事项、完成标准	持续保留
当前计划与未完成步骤	保留最新状态
最近的工具调用和结果	在单条上限内保留必要细节
较早的探索步骤	压缩成结构化事实
完整源码、日志和差异	保存引用，需要时再读取

代码块

```
目标：修复会话到期判断，保持现有测试不变。

已确认：
- 相关实现：src/auth/session-policy.ts
- 第一次测试：4 过、3 失败
- 日志：artifacts/runs/run-001/test-01.log

当前判断：
- 疑似与日期比较粒度有关，尚未通过修复验证。

下一步：
- 核对时间比较逻辑，提出补丁。
```

- 摘要保留：已做动作、执行结果、证据位置、未确认项。
- 已确认事实与诊断假设分别表达。
- 保留未解决失败及关键错误位置。
- 区分待审批、已执行与结果不明。
- 测试证据关联工作区版本或内容指纹。

- 工具调用与结果保持配对。
- 压缩后仍超预算，停止请求并记录原因。

5.4 验收：上下文受控，任务仍能继续

验收场景	要证明什么
测试输出大量日志	模型收到的观察有长度上限，完整日志仍可查回
失败信息出现在日志中间	关键失败被保留，或模型能通过引用补查
连续执行多轮搜索和读取	较早步骤被压缩，每轮组装后的输入受预算约束
压缩后继续修复	目标、限制、未解决错误和当前步骤仍然明确
测试通过后再次修改文件	旧测试结果不会被当作当前版本的完成证据
达到任务累计预算	系统按预算停止，并保留状态和停止原因

- 记录原始输出长度、模型观察长度与压缩情况。
- 请求前估算 Token，响应后累计实际用量。
- 字符数用于文本上限，不能替代精确 Token 数。
- 真实模型任务验证：截断或压缩后仍能查证、修复与复测。

六、按用户旅程安排人工确认

- 确认三个问题：在哪里暂停、展示什么、批准后谁执行。
- 授权范围内连续探索，具体变更按策略确认。

6.1 代码理解任务中，哪些操作不需要拦截

任务阶段	操作	本项目建议的确认方式
了解目录	list_files、read_directory	在授权范围内自动执行
查找入口	search_code	在授权范围内自动执行
阅读实现	read_file	在授权范围内自动执行，受敏感路径限制
生成说明	在指定产物目录创建新文档	满足预授权条件时自动执行
覆盖已有说明	修改已存在的文档	展示差异后确认
改动业务源码	超出代码理解任务的写入	按任务权限拒绝，需另行授权

- 自动执行仍经过校验、权限与沙箱检查。

- 指定产物目录允许新建，文件已存在则拒绝覆盖。
- 审批结合任务、路径与写入方式判断。
- 禁止仅按 `write_file` 工具名整体放行。

6.2 文件修改与命令执行，分别怎样审批

操作	审批前展示	执行前再次检查
应用补丁	文件路径、完整差异、修改理由	补丁是否一致，目标文件是否变化
写入文件	路径、写入内容、创建或覆盖方式	是否仍在授权范围，覆盖条件是否满足
运行测试	实际测试命令、工作目录、超时	测试配置及相关脚本是否变化
运行通用命令	可执行程序、参数、工作目录、执行范围	命令是否允许，审批是否仍有效

- 补丁审批绑定任务、工作区、补丁与修改前状态。
- 内容或目标文件变化，重新核对。
- 测试审批展示最终命令与脚本影响范围。
- 本地路径：显式领取并执行。
- HTTP 路径：等待任务收到批准后继续。
- 同一审批单只允许一条路径领取。
- 拒绝、过期、服务不可达均保留明确状态。
- 未获有效授权，不产生副作用。

6.3 审批时让人看什么证据

信息	帮助审批人判断什么
用户目标	这次操作是否服务于当前任务
失败证据	为什么需要修改，依据是哪次测试或日志
目标文件与补丁	具体改了什么，是否超出范围
当前验证状态	哪些检查已完成，哪些要执行后才能验证
影响范围	是否涉及源码、测试、依赖或构建配置
操作编号与有效期	批准的是哪一次操作，授权何时失效

- 执行记录提供退出码、差异与操作状态。
- 模型解释修改理由，区分事实与推断。

- 审批反馈进入下一轮，修改后的方案重新审批。
- 批准表示允许执行，复测决定是否完成。
- 完整过程：提出 → 展示 → 决定 → 执行 → 验证。

6.4 用两类任务生成自动评估报告

评估项	代码理解	测试修复
输入	登录逻辑分析要求	会话判断修复要求
初始环境	指定版本的干净样本	保留已知缺陷与失败测试的样本
预期产物	登录说明文档	源码差异、复测结果、变更说明
修改范围	指定报告文件	允许修改的源码
关键检查	引用路径有效，原文件未改	目标测试通过，现有测试未被削弱
人工确认	按报告创建或覆盖策略	按补丁与命令策略
当前覆盖限制	内容准确性需要进一步核对	新增测试的用例有效性需单独核对

- 使用独立工作副本与独立运行记录。
- `cli eval` 汇总两类任务。
- 待审批、拒绝与完成分别记录。
- 自动生成报告不自动批准。
- 测试审批器与真实人工审批分别标注。

字段	用途
任务名称、Run ID、决策器来源	确认运行对象与执行方式
执行状态与停止原因	区分完成、失败、待审批、取消和预算耗尽
验收结果	区分程序结束与交付条件通过
工具调用与审批次数	观察执行过程和人工参与量
耗时、Token 与费用	观察资源消耗
产物、日志与差异引用	支持复核
未覆盖能力与待复核项	明确当前结论的范围

- 执行状态与验收结果分别判断。
- 文档不存在、越权修改、削弱测试均不能通过。
- 路径有效可自动检查，解释准确性需对照源码。

七、组合完整项目：从模块到运行闭环

- 明确模块交接：输入、检查、执行、保存、回写。
- 沿一次任务检查完整责任链。

7.1 项目目录与模块职责

已有位置	职责
src/cli.ts	接收任务、处理命令行操作、查询结果
src/pi/provider.ts	连接模型
src/pi/coding-agent-extension.ts	从 pi 扩展入口接入受控工具能力
config/platform.yaml	平台级模型、预算和执行边界
config/tasks.yaml	各类任务的目标与执行策略
hitl/service.py	承担 LangGraph 审批流程
data/platform.db	保存本地审批、审计等运行记录
../samples/	存放被分析和修改的样本仓库

模块职责	接收什么	交付什么
任务装配	用户目标、任务类型、工作区	本次任务的有效配置与运行上下文
计划管理	目标、执行观察、当前进度	当前计划、修订记录与待完成步骤
模型与决策适配	经过预算处理的上下文、工具定义	工具调用或结束建议
Tool Runtime	工具调用、用户身份、当前策略	校验、授权和执行后的结构化结果
仓库工具	已通过检查的参数	文件、命令和 Git 操作结果
上下文处理	历史步骤、原始工具结果	下一轮需要的观察、摘要和证据引用
状态与 Checkpoint	任务进度、待处理操作、执行记录	可恢复的任务状态
完成检查与评估	当前产物、测试结果、差异	验收结论与报告

- pi：模型与工具循环。
- LangGraph：当前审批流程。
- 任务运行层：依据审批与执行结果继续、暂停或结束。
- 按职责定位问题，沿用已有目录与接入点。

7.2 任务进入、计划执行、工具调用与状态更新

第一步，接收任务并建立运行上下文。

- 加载平台与任务配置，检查工作区，生成 Run ID。
- 固定预算、工具范围与读写权限。

- 记录真实模型或离线脚本来源。

```
任务：以 project2 为基础，实现「接收任务并建立运行上下文」——一句话目标和任务类型进来，装配出这次运行的有效策略。

【涉及文件】
  src/cli.ts          入口：接收目标、--task、--dir 与权限参数
  src/platform.ts     buildPlatform：装配 config / sandbox / store / registry / policy / runtime
  src/config.ts       loadConfig：读 config/platform.yaml，HITL 参数可从环境变量覆盖
  src/tasks.ts        resolveTask / applyTask：按 --task 或关键词路由，再做配置合并
  src/sandbox.ts      按合并后的 sandbox 建边界

【做到】
  优先级固定：platform.yaml 默认值 < 任务条目 < 命令行 --dir
  生成 Run ID 并落库；确认本次动作来自离线脚本还是真实模型，写进运行记录
  未知任务类型、或目标不匹配任何条目时直接报错，不默默兜底

【验收】
  不带 --task 时按关键词路由到对应任务；--dir 覆盖任务里的 workspace
  路由失败抛错，错误信息指出可选任务类型
  一次运行的所有模块共享同一份运行上下文（预算、工具范围、读写权限）
```

第二步，建立计划并准备模型输入。

- 计划记录步骤与完成条件。
- 组装目标、限制、计划与观察，检查输入预算。
- 离线动作与真实模型共用后续执行路径。

```
任务：以 project3/v2 为基础，实现「建立计划并准备模型输入」——把任务简报、工具定义与历史观察组装成受预算约束的一轮输入。

【涉及文件】
  src/pi/prompt-client.ts    渲染系统提示词（读 config/prompts.yaml；gateway 与 local 两个来源）
  config/prompts.yaml        任务简报、交付物等变量模板
  src/context-budget.ts      组装前检查输入预算

【做到】
  提示词带上任务简报与交付物路径：领了任务就按那一类说话
  本轮工具定义与执行校验用同一份来源（模型看到的参数要求 = Runtime 校验的要求）
  请求前检查整体输入预算，超限则停止本轮并说明原因
```

```
12
13  【验收】
14    prompts.yaml 改一行，下一轮提示词跟着变；离线脚本路径与真实模型共用同一份上下文
15    输入超预算时本轮不发请求，停止原因可查
```

第三步，接收工具调用，进入 Runtime。

- 参数校验 → 权限 → 预算 → 审批。
- 待审批操作先保存，获批后复核内容与条件。

代码块

```
1   任务：以 project3/v2 为基础，实现「接收工具调用，进入 Runtime」——模型提出的调用，必须先
    过校验、权限、预算和审批四道检查。
2
3   【涉及文件】
4     src/contract.ts      contractRulesFor / verifyContracts：参数契约与补充规则
5     src/policy.ts        可见性与权限判断
6     src/registry.ts      工具注册与查询
7     src/runtime.ts       invoke / gateForPi：校验 → 权限 → 预算 → 审批
8     src/loop-guard.ts    调用预算与重复调用护栏
9
10  【做到】
11    参数不合法不得进入 handler，返回结构化错误并保留 toolCallId
12    需要审批的调用先保存待执行操作，进入等待状态；未批准不得写入
13    同一张审批单只能被一条执行路径领取，重复通知不产生重复副作用
14
15  【验收】
16    read_file 的 path 传成数字：handler 调用次数为 0
17    待审批写入未批准时文件保持不变；批准后执行的正是获批的那次操作
```

第四步，在执行边界内调用工具。

- 在受限路径与命令范围内执行。
- 返回退出码、错误、超时状态与证据引用。

代码块

```
1   任务：以 project3/v2 为基础，实现「在执行边界内调用工具」——十件仓库工具只在沙箱允许的范
    围内生效。
2
3   【涉及文件】
4     src/sandbox.ts     路径边界 check、deny_read / deny_write、命令与输出上限
5     src/tools/*.ts     十个工具的 handler（fs-read / fs-write / exec / git）
6
7   【做到】
```

```
    所有文件路径先过沙箱边界；子进程环境剔除疑似密钥变量
    命令执行带超时与强制结束；单次输出按字符上限截断
    结果保留退出码、错误类型、是否超时和必要的文件/日志引用

【验收】
    ../../outside.txt 这类路径被边界拒绝，错误来自正确的检查层
    超时命令被结束，结果里 timed_out 为 true
```

第五步，保存结果并更新进度。

- 关联调用编号，保存审计与恢复信息。
- 依据执行事实更新状态与计划。
- 受控观察进入下一轮。
- 测试进程结束与修复通过分别判断。

代码块

```
任务：以 project3/v2 为基础，实现「保存结果并更新进度」——结果入库、上下文受控地进入下一轮。

【涉及文件】
  src/store.ts            审计与 Trace 落库（platform.db）
  src/context-budget.ts   clipForAudit（入库截断 + 全量哈希）、renderContext（历史压缩）
  src/loop-guard.ts       记账：turns / toolCalls / tokens / cost

【做到】
    结果与原调用编号关联；审计保存“发生过什么”，不保存全文
    进模型的观察有长度上限，完整结果留在产物里可回查
    较早步骤压成一行摘要，保留目标、未解决失败和证据引用

【验收】
    长日志入库被按行截断且带 full_sha256；模型看到 truncated 标记
    第三轮时第一步原文不在上下文里，只剩一行摘要；调用与结果配对完整
```

第六步，检查继续还是结束。

- 模型提出结束建议。
- 程序核对产物、当前测试与改动范围。
- 证据不足时继续或明确停止。

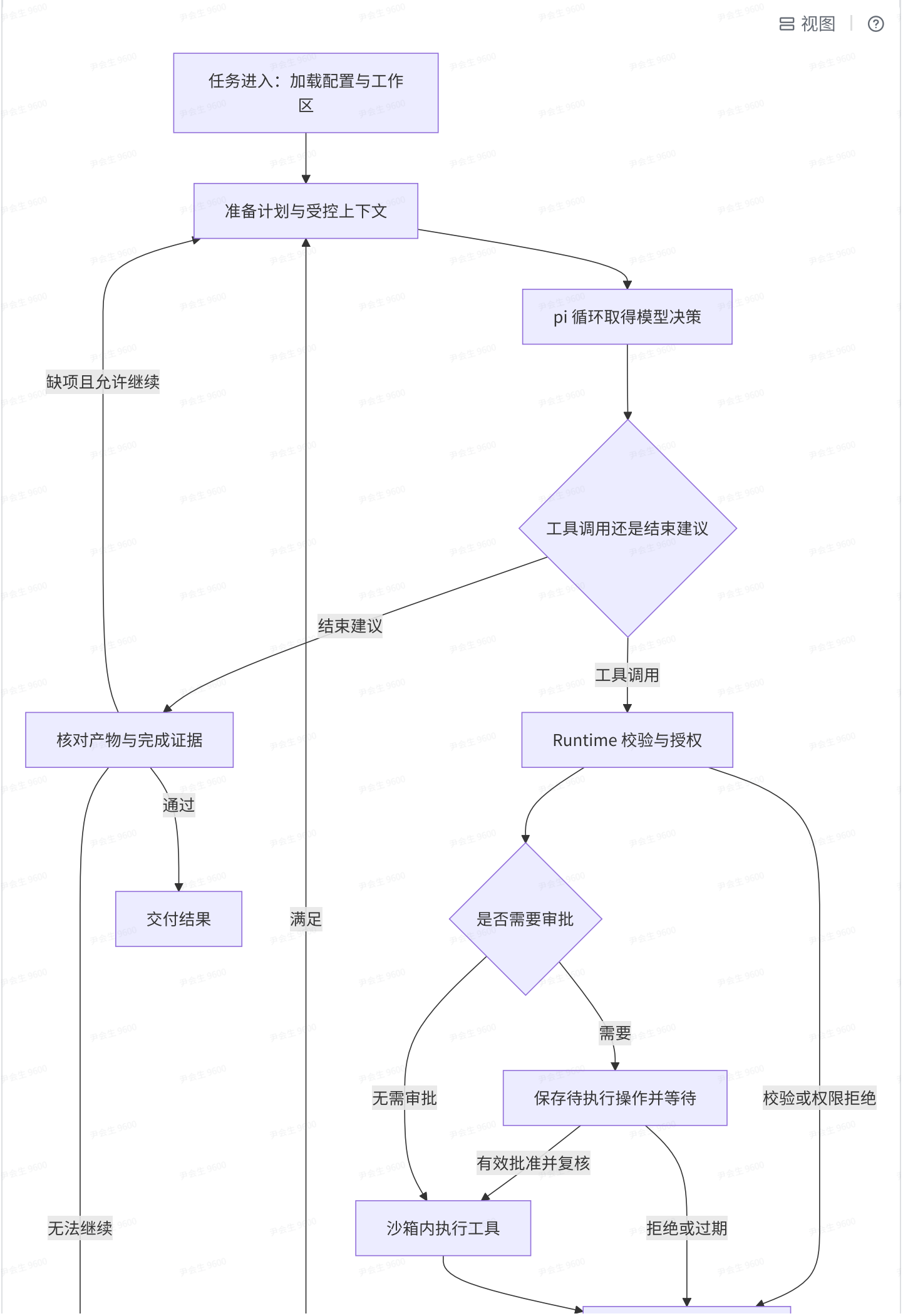
代码块

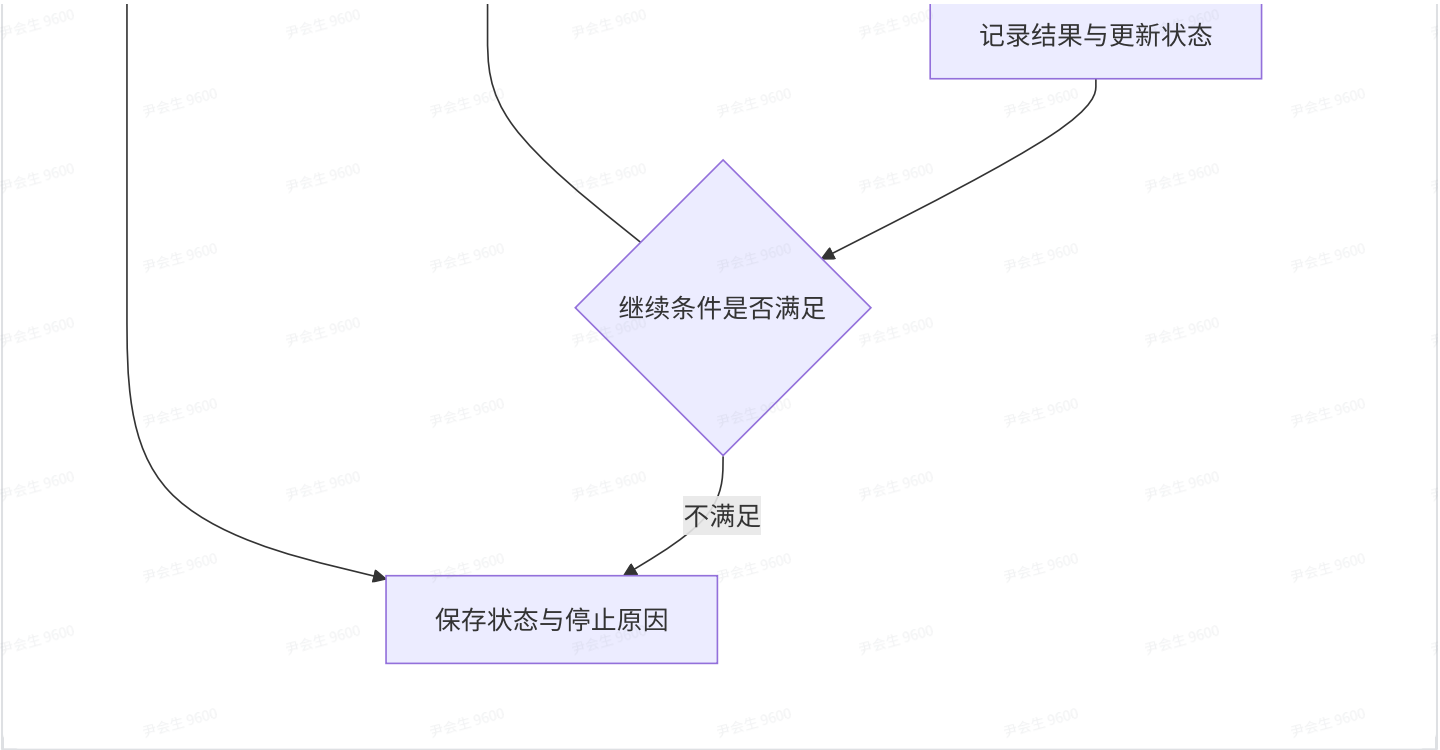
```
任务：以 project3/v2 为基础，实现「检查继续还是结束」——结束不等于完成，证据齐了才算交付。

【涉及文件】
  src/deliverable.ts        交付物核对：写工具落盘、成功写入、未被覆盖
  src/loop-guard.ts         预算耗尽时给出停止原因
  src/pi/agent-runner.ts    Loop 收口与最终状态

【做到】
  模型提出结束时，核对交付物路径、测试结果和改动范围，缺什么报什么
  证据不足时受剩余预算约束地继续，或停止并报告未完成
  区分“执行状态”与“验收结果”；没有模型调用的测试标“不适用”

【验收】
  Agent 正常结束但交付物不存在：验收判失败，状态与原因可查
  预算耗尽时停止并保留停止原因；未完成任务不会被计为完成
```


视图
任务进入：加载配置与工作区
准备计划与受控上下文
pi 循环取得模型决策
工具调用还是结束建议
结束建议
工具调用
缺项且允许继续
核对产物与完成证据
Runtime 校验与授权
通过
交付结果
满足
是否需要审批
需要
无需审批
保存待执行操作并等待
校验或权限拒绝
有效批准并复核
沙箱内执行工具
拒绝或过期
无法继续



7.3 沙箱、Checkpoint、审计和人工确认怎样协作

能力	回答的问题
人工确认	这次具体操作是否获得授权?
Sandbox	操作实际发生时，能访问和影响哪些资源?
Checkpoint	任务暂停以后，继续执行需要哪些状态?
审计	刚才提出、批准和执行了什么，有什么结果?

- 保存路径、补丁、修改前状态与审批编号。
- 批准后复核权限与现场，再执行并更新记录。
- Run ID、调用编号、审批编号与产物引用统一关联。

中断位置	恢复时怎样处理
仍在等待审批	查询原审批状态，核对有效期和待执行内容
已批准，能够确认尚未开始执行	重新检查执行条件，再按既有领取机制执行
操作可能已经发出，结果未保存	先核对文件或进程现场，确认实际后果
结果已持久化，但计划未更新	根据已有结果补齐状态，避免再次执行
补丁已应用，但尚未复测	继续验证当前内容

- Checkpoint 保存状态，不能撤销文件修改。
- LangGraph Checkpoint 恢复审批流程；完整任务另需保存计划与执行证据。

- 结果不明先核对现场，无法确认则人工接管。

7.4 运行两类任务并核对最终产物

- 确认真模型、初始工作区与审批服务就绪。
- 每次验收使用独立工作副本。

任务一：代码理解

代码块

```
node --import tsx src/cli.ts agent \
  "帮我找出该项目的登录逻辑，并生成 Markdown 说明文档。" \
  --task understand \
  --dir ../samples/1_login_demo
```

产物或记录	核对内容
登录说明文档	入口、调用关系和关键判断是否准确
路径引用	是否指向实际存在且相关的代码
Git 状态与差异	是否只产生允许的文档改动
Trace	文档中的关键结论是否有阅读依据
最终状态	文档是否实际落盘，是否仍有待执行操作

任务二：测试修复

代码块

```
HITL_API_KEY=demo-key node --import tsx src/cli.ts agent \
  "读取登录会话模块的失败测试与日志，修复源码并复测，不修改现有测试。" \
  --task fix_tests \
  --dir ../samples/2_bug_fix/demo-app
```

产物或记录	核对内容
初始失败日志	修复前的问题是否确实存在
源码差异	是否针对失败原因，是否存在无关修改
最终测试结果	是否对应修改后的当前内容
审批记录	实际执行内容是否与获批内容一致
变更说明	是否准确说明修改与验证结果

- 独立复测，检查 Git 差异。
- 评估报告汇总状态、产物、审批与资源消耗。

7.5 提示词：生成整套工程

代码块

```
任务：以 agent-tool-runtime-pi-v2（阶段二的工具运行时）为基础，生成 project3/v2——把第三章的 Loop、Planning、Checkpoint、Sandbox、Harness 与 LangGraph 审批组合成一台 Codebase Agent：十件仓库工具在沙箱边界内执行，参数先过契约校验，敏感操作过人审批，上下文受预算约束，代码理解与测试修复两类任务开箱可跑。

【工程目录】
project3/
  samples/1_login_demo/          代码理解样本（只读，只写一份文档）
  samples/2_bug_fix/demo-app/    测试修复样本（跑测试、只改 src）
  v2/
    src/
      cli.ts                 命令行入口
      config.ts              平台配置加载与类型
      platform.ts            装配 config / sandbox / store / registry / policy / runtime
      tasks.ts               任务解释器（tasks.yaml → TaskConfig）
      domain.ts              领域类型（ManagedTool / ToolRuntimeError / ExecutionContext）
      registry.ts            工具注册表
      policy.ts              可见性与权限判断
      contract.ts            参数契约（补充规则 + 渲染）
      runtime.ts             工具执行：校验 → 权限 → 预算 → 审批 → 执行
      sandbox.ts             工作区边界：路径与拒绝名单、命令、超时
      store.ts               本地账本（SQLite）
      loop-guard.ts          预算与重复调用护栏
      context-budget.ts      上下文渲染与压缩
      deliverable.ts         交付物核对
      name-mapping.ts        内部名 ↔ 模型可见名
      tools/                 args / clip / fs-shared / fs-read / fs-write / exec / git / index
      pi/                    provider / prompt-client / decider / agent-runner / coding-agent-extension
      hitl/client.ts         审批 HTTP 客户端
    config/
      platform.yaml          模型、预算、沙箱、策略、审批的公共约束
      tasks.yaml             任务知识（每类任务一个条目）
      prompts.yaml           系统提示词模板
      scripts/*.yaml         离线决策脚本（可逐字重跑）
```

```
    hitl/
      service.py          LangGraph 审批服务（FastAPI）
      graph.py            审批图（谁能批、能否自批）
    tests/unit/           单测，每件能力一份 kg_*.test.ts
    data/platform.db      运行账本（首次运行自动创建）

【依赖与环境】
  Node >= 22.19;
  TypeScript 用 tsx 直接跑：node --import tsx src/cli.ts
  运行时依赖：@earendil-works/pi-agent-core / pi-ai / pi-coding-agent（0.87.1）、commander、typebox、yaml
  开发依赖：tsx、typescript、vitest;
  Python 侧见 hitl/requirements.txt

【配置字段规格】
  platform.yaml
    gateway: base_url / api_key_env / prompt_name / prompt_version
    prompt: source(local|gateway) / file
    model: provider_id / id / name / context_window / max_tokens / 三个单价
    runtime: database / max_turns / max_tool_calls / max_repeated_call / max_total_tokens / max_cost_usd / max_duration_ms / context{keep_recent_turns,audit_max_lines,audit_max_chars}
    hitl: mode(disabled|http) / gate(auto|none|工具名单) / base_url / api_key_env / approvers / timeout_ms / wait_timeout_ms
    policy: role_permissions / agent_tool_allowlist
    sandbox: root / deny_read / deny_write / max_read_chars / max_output_chars / max_capture_bytes / command_timeout_ms / kill_grace_ms / secret_env_pattern
    tools: test_command（没领任务时的兜底）
    tasks: file
  tasks.yaml（每类任务一个条目）
    display_name / match[] / workspace / script / brief / deliverable / test_command / budget{max_turns,max_tool_calls} / deny_write_extra[] / approval{gate,via,approvers}
  prompts.yaml
    required_variables + template; 变量: tenant_id / user_id / agent_name / task_name / task_brief / deliverable
  scripts/*.yaml
    steps[].text + steps[].tool_calls[].name/args

【十件工具的契约】
    工具输入应说明什么输出应包含什么
    list_files相对目录、匹配条件、数量上限文件路径列表，是否截断
    search_code搜索内容、搜索范围、结果上限命中文件、行号、片段，是否截断
    read_file文件路径、可选的起始行与读取范围文件内容、实际行号范围，是否截断
    read_directory目录路径当前目录的直接子项及类型
    write_file文件路径、写入内容、覆盖规则写入路径、创建或覆盖结果
    apply_patch目标文件、补丁内容应用结果、变更摘要；失败时说明匹配问题
```

```
    run_test允许的测试目标或配置标识退出码、测试摘要、输出日志，是否超时
    run_command命令及参数、工作目录、超时退出码、标准输出、标准错误，是否超时
    git_diff可选的路径范围实际差异、涉及的文件，是否截断
    git_status可选的路径范围已修改、未跟踪等文件状态
    参数用 typebox 定义，执行前统一经过 contract 校验。
    risk 分三档（read / write / execute）：read 自动执行，write 与 execute 按任务策略进审批。

【CLI 子命令】
  tools / contract / tasks / agent / approvals（list|tickets|decide|execute）/ audit / trace
  agent 支持 --task / --dir / --script / --decider(real|auto|mock) / --roles

【本地账本】
  data/platform.db 三张表：audit_records（谁调了什么、结果与耗时）、trace_events（每轮上下文渲染等事件）、approvals（待批与已批的写操作）。

【验收 AC】
  AC-1 npm test 全绿：每件能力至少一份单测（契约 / 运行时 / 沙箱 / 工具 / 任务 / 审批 / 上下文 / 决策器 / 护栏 / 提示词）
  AC-2 任务一离线跑通：--decider mock 走 understand-login.yaml，停在写文件审批，批准并执行后生成 artifacts/login-flow.md
  AC-3 任务二离线跑通：run_test 第一步即被门口拦下（AWAITING_APPROVAL），批准后打补丁、复跑、git_diff
  AC-4 参数错误的调用不进入 handler；越界路径被沙箱拒绝
  AC-5 长输出被截断、旧步骤压成一行摘要，审计留全量哈希

【明确不做】
  - 不做跨任务批量评估入口（cli eval 留给后续）
  - 不接 MCP：十件工具走 pi 的 registerTool 扩展点
  - 不修改样本已有测试来让它变绿：tests/** 由沙箱拒绝写入
  - 不引入向量召回与长期记忆：短期记忆就是受预算约束的对话
```

八、验收与工程复盘

- 结果验收：文档、测试与差异。
- 过程验收：权限、审批、状态与恢复。
- 成功路径与失败场景共同验证。

8.1 按项目目标逐项验收

项目目标	验收动作	通过依据
Agent Loop	运行需要多次工具调用的任务	工具结果进入下一轮，任务能继续推进并正确停止
Planning	查看计划及执行中的修订	步骤有完成条件，进度与实际证据一致
搜索与读取	执行登录逻辑分析	能定位相关文件，说明中的引用真实有效
文件修改	执行受控写入和补丁操作	实际差异与任务要求、获批内容一致
测试与修复	从已知失败状态运行修复任务	保留修复前失败、源码差异及修复后通过结果
新增测试	为目标函数新增边界用例	新测试能暴露目标缺陷，修复后与原测试共同通过
人工确认	分别执行批准与拒绝场景	未批准不执行，批准只授权对应操作
Sandbox	尝试访问禁止路径或执行禁止操作	实际访问被阻止，拒绝原因可追踪
状态与 Checkpoint	在指定位置中断并恢复	找回进度，核对现场，避免重复写入
工具契约	提交缺字段、错类型等参数	handler 未执行，错误准确回写
上下文与预算	使用长日志和多轮任务	输入受控，关键证据可查，预算耗尽时停止
评估与追踪	汇总两类任务的执行记录	状态、验收结论、产物和消耗能够相互对应

- 标记：通过、失败、待验证、未覆盖。
- 每项附证据位置。
- 预置失败测试验证修复，不能证明测试生成能力。
- 证据充分即进入下一项；失败则定位缺口。

8.2 失败案例：参数错误、测试失败、审批中断与恢复

- 统一复盘：现象 → 执行事实 → 处理规则 → 修复证据。

案例一：参数错误，工具有没有提前执行？

- `write_file` 缺少 `content`。
- 预期：参数错误、handler 不执行、文件不变。
- 错误携带调用编号进入下一轮。
- 修复验收：非法参数零调用；合法参数完整检查后正常执行。

案例二：测试失败，系统有没有把执行结束当成任务完成？

- 保留失败退出码、断言与日志。
- 测试进程结束不能直接标记修复完成。

观察到的现象	优先检查
模型没有看到失败日志	工具结果回写、截断和上下文组装
模型看到了失败，计划却标记完成	计划完成条件
模型提出结束，程序直接通过	完成检查
修改后沿用旧的通过结果	测试证据与工作区版本的关联

- 修复对应程序检查，再用原失败场景复验。
- 提示词要求不能替代完成条件验证。

案例三：审批中断，恢复后会不会错用旧授权？

- 找回原待审批操作与审批状态。
- 核对有效期、补丁、文件状态与授权条件。
- 目标文件变化，重新核对方案。
- 重复批准或重复执行不能重复写入。
- 记录领取状态、完成状态与结果。

案例四：文件已经修改，执行结果没有保存

- Checkpoint 无成功记录，不能推断尚未执行。
- 已应用：确认现场、补记事实、继续测试。
- 未应用：复核条件后处理。
- 无法判断：保留现场，人工接管。
- 核心要求：状态与执行事实一致，避免盲目重放。

8.3 本次实现的边界与已知限制

边界	应怎样说明
样本覆盖	两类样本通过，只能证明对应任务与环境下的表现
新增测试	已放开受控写入（tests/generated/）；新用例是否有效仍需独立证据
配置化	已有工具能够承接的任务可以复用配置；新能力仍需扩展代码
模型输出	同一目标可能产生不同操作顺序和消耗，需要按结果与约束验收
上下文压缩	可能省略细节，必须保留关键事实和补查入口
中断恢复	对结果不明的写操作，需要核对现场，部分情况需人工接管
Sandbox	保护范围取决于实际执行机制，路径检查不能代替进程层隔离
自动评估	文件、测试和差异可自动核对（cli eval 已接入）；解释准确性仍需明确检查点或人工复核

- 已补齐：`cli eval` 与 `allow_write_extra`。
- 待接入：严格真实模型模式与部分上下文规则。
- 已完成能力必须有代码与验收记录支持。
- 单次成功是样本证据；稳定性需要固定任务集与多次记录。

8.4 哪些问题由 AI 辅助，哪些必须由工程师把关

工作	AI 可以辅助	工程师必须把关
需求拆解	整理流程、提出任务与模块划分	交付目标、完成条件和范围是否正确
代码组合	查找接入点、生成适配代码	是否复用已有路径，是否绕过关键检查
工具契约	编写 Schema 和错误结构	输入输出是否足够明确，失败是否阻止执行
人工确认	实现审批流程与展示内容	授权绑定什么，何时失效，如何避免重复执行
状态恢复	编写存档和恢复分支	状态是否符合现场，写操作能否安全继续
测试验收	生成用例、执行命令、汇总记录	用例是否验证目标，结果是否足以支持结论
工程复盘	整理 Trace、提出原因假设	依据是否成立，修复是否处理了真正原因

- 能解释：Loop、Runtime、Harness、Sandbox、Checkpoint 的职责。
- 能追踪：输入 → 决策 → 校验 → 执行 → 回写 → 完成检查。
- 能修复：未校验执行、旧审批复用、结果丢失、写操作误重放。
- 能验证：用测试证明关键边界有效。
- 学习重点：理解关键连接，指导实现，判断交付证据。

8.5 提示词： 按复盘结论优化工程

代码块

```
任务：以 project3/v2 为基础，做一轮验收驱动的优化，把已知限制补成能力。按优先级逐项做，每改一项都补齐单测，npm test 必须全绿，默认配置下的对外行为保持不变。

【优化项 1：受控新增测试（最高优先）】
  现状：tasks.yaml 的 fix_tests 用 deny_write_extra: ["tests/**"] 把测试目录整体封死，
        于是“补充测试”这个任务目标无法完成，只能修源码。
  目标：允许 Agent 在受控位置新增测试，同时保护已有测试。
  要求：
    - sandbox 增加“允许写入”例外：命中 deny_write 但在 allow_write 名单内的路径放行；默认为空，不影响既有行为
```

```
    - tasks.yaml 的 fix_tests 增加 allow_write_extra: ["tests/generated/**"], deny_write_extra 保持 ["tests/**"]
    - 样本 package.json 的 test 脚本改为匹配子目录（tests/**/*.test.ts）：这属于样本准备，不由 Agent 执行
    - brief 写明：新增测试只写到 tests/generated/，已有测试一行不改；靠改测试让红用例变绿仍被拒绝
    - 验收：新增测试路径可写；tests/ 下已有文件仍被拒绝；run_test 能跑到新增测试

【优化项 2：评估入口 cli eval】
  现状：README 与 8.3 都说明没有统一评估入口。
  目标：新增 cli eval，依次跑两类任务并汇总报告。
  要求：
    - 每类任务使用独立工作副本与独立账本，避免互相污染
    - 报告字段：任务名 / Run ID / 决策来源 / 执行状态与停止原因 / 验收结论 / 工具调用与审批次数 / 耗时与 token / 产物与差异引用 / 未覆盖项
    - 待审批任务记为“待审批”，不得计为完成；自动生成报告不等于自动批准
    - 输出 JSON 与 Markdown 两份（artifacts/eval_report.md）
    - 验收：两类任务各跑一次，报告字段齐全，失败可定位

【优化项 3：提示词变量清单】
  prompts.yaml 的 required_variables 只列了 tenant_id / user_id / agent_name，
  模板实际还用了 task_name / task_brief / deliverable——补齐清单，缺变量时报错，而不是静默留占位符。

【优化项 4：清理遗留配置】
  config/gateway.yaml 在 v2 里没有任何引用（提示词来源是 local），确认后删除，避免“两处配置”的误读。

【优化项 5：任务一样本口径】
  scripts/understand-login.yaml 没有读 src/middleware/auth-middleware.ts，
  与“阅读路由、中间件、Service、数据库”的说明不一致——补一次 read_file，或在文档中标注中间件为可选。

【通用要求】
  - 每项改动都补/改单测；不改现有测试的断言来“绕过”新行为
  - sandbox 默认配置下行为与现在完全一致，新字段默认为空
  - 改动同步到 README（新增字段与命令）
```

- 执行顺序：P6 基础验收 → P7 补齐限制。
- 文稿记录的落地结果：允许写入例外、评估入口、变量校验、配置清理、中间件补读。
- 文稿记录的单测数量：117 → 121。

九、本节小结：从机制学习走向项目交付

- 阶段二：建立受控工具执行基础设施。
- 阶段三：连接 Loop、计划、状态、沙箱、Harness 与审批。
- 完整任务链：理解目标 → 执行操作 → 观察结果 → 处理偏差 → 验证完成。
- 项目完成度由产物、测试、差异与执行记录支持。
- 工程师核心能力：拆解目标、识别边界、组合模块、验证失败、持续演进。